오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

Bootstrap 5

DAY 08

# Bootstrap 5 개념과 기본 사용법

미리 만들어진 클래스로 화면을 빠르게 구성합니다.
1챕터에서 만든 페이지를 Bootstrap으로 다시 만들어 봅니다.

강사 김태민

TODAY'S GOAL

# 오늘 끝나면 할 수 있는 것

1. **Bootstrap을 페이지에 적용할 수 있다**

   CDN 한 줄로 연결하고 적용됐는지 확인합니다.

2. **Grid로 레이아웃을 구성할 수 있다**

   container, row, col과 반응형 접두사를 씁니다.

3. **컴포넌트와 유틸리티로 UI를 빠르게 만들 수 있다**

   Navbar, Card, Form, Modal과 간격·색 클래스를 씁니다.

### 오늘의 결과물

Bootstrap으로 다시 만든 소개 페이지 — 코드는 짧아지고 화면은 정돈된 상태

ROADMAP

# 오늘의 흐름

## 01

### Bootstrap이란

CSS 프레임워크
CDN 적용

## 02

### Grid System

12칸과 반응형
실습 01

실습 포함

## 03

### 컴포넌트

Navbar·Card·Modal
실습 02

실습 포함

## 04

### Utility Class

간격·색·정렬
언제 쓰나

앞의 두 섹션은 뼈대를 잡는 법, 뒤의 두 섹션은 살을 붙이는 법입니다.
오늘 만든 페이지가 다음 시간 미니 프로젝트의 출발점이 됩니다.

되짚기 · 01

# 지금까지 CSS를 전부 직접 썼습니다

```
/* style.css — 1챕터에서 직접 쓴 규칙 */
* { box-sizing: border-box; margin: 0; padding: 0; }
.nav   { display: flex; justify-content: space-between; }
.menu  { display: flex; gap: 16px; list-style: none; }
.cards { display: grid; grid-template-columns: 1fr; }
.card  { padding: 20px; border: 1px solid #d3dee6; }
@media (min-width: 768px)  { … }
@media (min-width: 1024px) { … }
```

### 이 방식의 좋은 점

원하는 대로 정확히 만듭니다.
무엇이 왜 그렇게 되는지
전부 설명할 수 있습니다.

### 힘든 점

새 화면마다 처음부터 씁니다.
버튼 하나에도 색과 여백,
모서리를 다 정해야 합니다.

# 같은 카드를 Bootstrap으로 쓰면

```
<!-- 직접 쓴 방식 : HTML + CSS 두 곳을 고칩니다 -->
<article class="card">…</article>
.card { padding: 20px; border: 1px solid #d3dee6;
        border-radius: 8px; background: #fff; }

<!-- Bootstrap : HTML 한 곳만 고칩니다 -->
<div class="card p-3 shadow-sm">…</div>
```

### 무엇이 달라졌나

CSS 파일을 열지 않고
클래스 이름만으로 모양을
정했습니다.

### 대신 잃는 것

클래스 이름을 외워야 하고
화면이 비슷비슷해집니다.

준비

# 오늘의 셋팅

```
oz-lecture/Day08/          ← oz-lecture 안에 만듭니다
  index.html               ← 새로 만듭니다
  style.css                ← 직접 쓴 규칙은 조금만 남깁니다
  me.jpg

$ git commit -m "DAY08 Bootstrap 실습"
/* 끝나면 add · commit · push — 오늘도 main 하나 */
```

### DAY 07과 이어집니다

- oz-lecture 저장소에서 작업합니다
- 오늘도 main에 바로 커밋합니다
- 끝나면 push합니다

### 인터넷이 필요합니다

CDN으로 불러오므로
연결이 끊기면 스타일이
사라집니다.

01

# Bootstrap이란 무엇인가

CSS 프레임워크 · CDN 적용 · 내 CSS와 함께 쓰기

01 · 개념

# 미리 만들어 둔 CSS 묶음입니다

직접 쓸 때

## 규칙을 내가 정합니다

여백을 20px로 할지 24px로 할지,
모서리를 얼마나 둥글게 할지
매번 정하고 매번 적습니다.

Bootstrap을 쓸 때

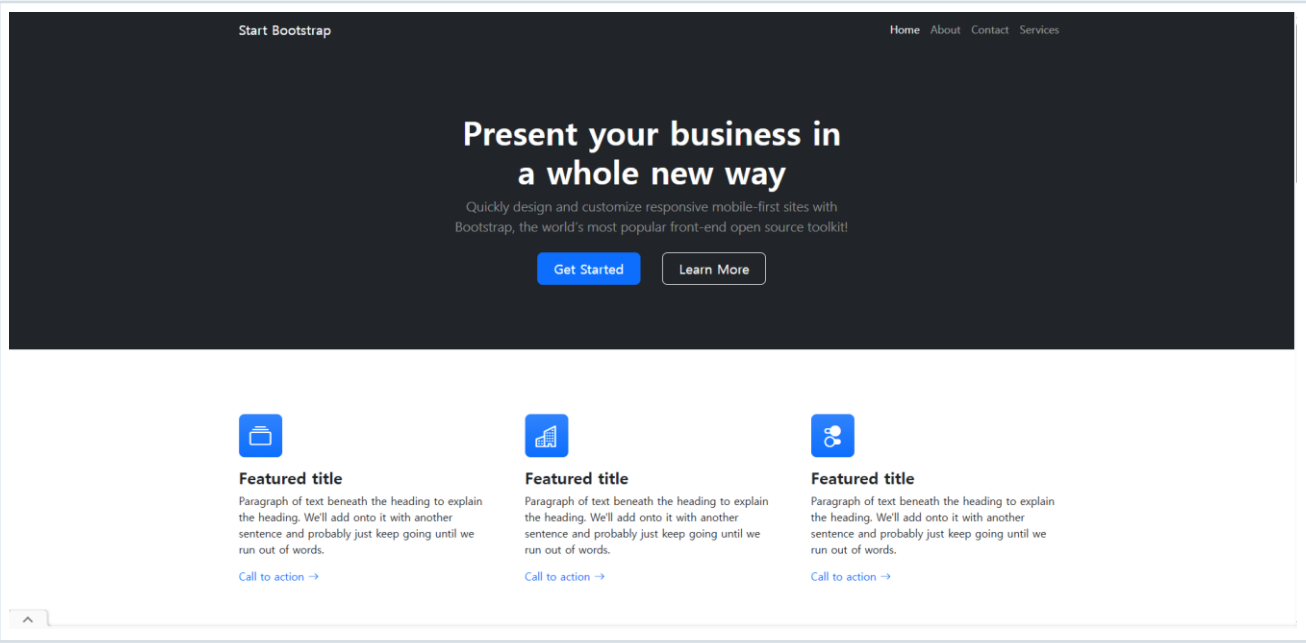
## 정해진 규칙을 가져다 씁니다

여백·색·크기가 이미 정해져 있고
클래스 이름으로 골라 씁니다.
화면 전체가 자동으로 일관됩니다.

**반응형과 모바일 퍼스트를 전제로 만들어진 오픈소스 CSS 프레임워크입니다**

내부가 Flexbox로 되어 있고, 5버전부터 jQuery 없이 동작합니다. 3·4버전 코드와 클래스 이름이 다릅니다.

# Bootstrap만으로 이런 화면이 나옵니다




Bootstrap 클래스만으로 만든 페이지 예시

**색과 여백, 버튼 모양이 이미 정해져 있습니다**

# Bootstrap이 주는 것은 세 가지입니다

01

## Grid System

화면을 12칸으로 나눠
배치하는 규칙입니다.
반응형이 기본입니다.

02

## 컴포넌트

버튼, 카드, 내비게이션 바,
모달처럼 자주 쓰는 덩어리를
미리 만들어 둔 것입니다.

03

## 유틸리티 클래스

여백, 색, 정렬처럼
한 가지만 바꾸는
작은 클래스들입니다.

**뼈대(Grid) → 덩어리(컴포넌트) → 마무리(유틸리티)**

01 · 적용

# CDN 두 줄이면 준비가 끝납니다

```
<head>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8
        /dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body>
  …
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8
          /dist/js/bootstrap.bundle.min.js"></script>
</body>
```

### CSS는 head, JS는 끝

JS는 모달이나 내비게이션 접힘처럼 움직이는 것에만 필요합니다.

### CDN은 인터넷에 있는 파일을 그때그때 불러오는 방식입니다

파일을 내려받아 프로젝트에 넣는 방법도 있지만 오늘은 CDN으로 합니다.

01 · 확인

# 적용됐는지 이렇게 확인합니다

```
<button class="btn btn-primary">눌러보기</button>

/* 파란 버튼 모양이 나오면 적용된 것입니다 */
/* 밋밋한 기본 버튼이면 연결이 안 된 것입니다 */

확인 순서
  1. 개발자 도구 Network 탭에서 css 파일이 200인지
  2. link 태그가 head 안에 있는지
  3. 주소에 오타가 없는지
```

### 글꼴부터 바뀝니다

연결되는 순간 페이지 전체의
글꼴과 기본 여백이 달라집니다.
그것도 확인 신호입니다.

**DAY 04에서 배운 방식 그대로입니다 — 안 되면 Network 탭부터 봅니다**

# 직접 쓴 CSS와 같이 쓸 수 있습니다

```
<head>
  <link href="…bootstrap.min.css" rel="stylesheet">
  <link href="style.css" rel="stylesheet">   ← 뒤에 둡니다
</head>

/* 나중에 적은 규칙이 이깁니다 — DAY 04 캐스케이딩 */
/* 그래서 내 style.css 를 아래에 둡니다 */

/* style.css */
.card { border: 0; box-shadow: 0 2px 12px rgba(0,0,0,.06); }
body  { font-family: "Pretendard", sans-serif; }

/* Bootstrap 기본값 위에 내 색과 글꼴을 덮어씁니다 */
```

### 같으면 나중이 이깁니다

Bootstrap을 먼저,
내 CSS를 나중에 둡니다.
반대로 두면 내 규칙이
덮여버립니다.

### 그래도 안 먹으면

Bootstrap 쪽 선택자가
더 구체적인 경우입니다.
Styles 탭에서 확인합니다.

# 여기까지 정리

| | | |
|---|---|---|
| 1 | **프레임워크** | 미리 만들어 둔 CSS 묶음을 클래스 이름으로 가져다 씁니다 |
| 2 | **구성** | Grid, 컴포넌트, 유틸리티 클래스 세 갈래입니다 |
| 3 | **적용** | head에 CSS 링크, body 끝에 JS 스크립트 |
| 4 | **함께 쓰기** | 내 style.css는 Bootstrap 링크보다 뒤에 둡니다 |

**다음은 화면을 나누는 Grid입니다**

02

# Grid System

12칸 · container·row·col · 반응형 · 실습 01

# 화면을 기본 12칸으로 나눕니다

```
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |

col-6  col-6                  → 반씩 두 칸
col-4  col-4  col-4           → 셋으로 균등
col-8  col-4                  → 본문과 사이드바
col-3  col-3  col-3  col-3    → 넷으로 균등


/* 숫자를 더해 12가 되면 한 줄에 들어갑니다 */
/* 12를 넘으면 넘친 칸이 다음 줄로 내려갑니다 */

col-8  col-8  →  8+8=16 이므로 두 번째가 다음 줄로
```

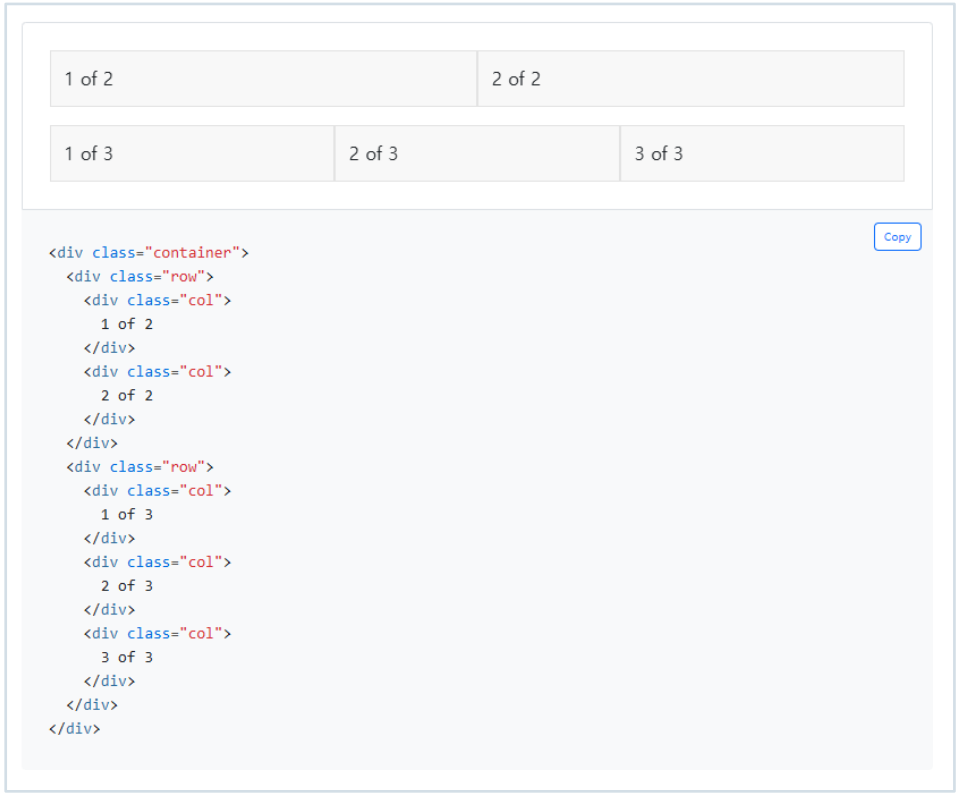
### 왜 12인가

2, 3, 4, 6으로 나누어떨어져
균등 분할이 쉽습니다.

### 12를 넘으면

넘친 칸이 다음 줄로
자동으로 내려갑니다.

# 숫자를 안 적으면 칸이 알아서 나뉩니다




col을 두 개 넣으면 6칸씩, 세 개 넣으면 4칸씩
자동으로 나뉩니다.

### 위쪽이 결과

한 줄이 같은 폭으로
나뉜 모습입니다.

### 아래쪽이 코드

container 안에 row,
row 안에 col을 넣은
구조가 그대로 보입니다.

# 세 겹으로 감쌉니다

```
<div class="container">
  <div class="row">
    <div class="col-4">첫 번째</div>
    <div class="col-4">두 번째</div>
    <div class="col-4">세 번째</div>
  </div>
</div>

/* container 안에 row, row 안에 col — 순서가 정해져 있습니다 */
```

### container

전체 폭을 잡고
가운데 정렬합니다.
끝까지 채우려면
container-fluid를 씁니다.

### row와 col

row는 한 줄,
col은 그 줄 안의 칸입니다.
col은 반드시 row 안에.

# container는 세 가지가 있습니다

```
<div class="container">
  화면 크기마다 정해진 폭으로 멈추고 가운데 정렬됩니다

<div class="container-fluid">
  화면 끝까지 채웁니다

<div class="container-md">
  768px 아래에서는 끝까지, 그 위에서는 폭이 고정됩니다
```

### 무엇을 고를까

본문은 container,
배경 띠나 상단 바는
container-fluid를 씁니다.

### 본문 폭을 제한하는 이유는 DAY 06에서 본 것과 같습니다

넓은 화면에서 글줄이 너무 길어지면 읽기 어려워집니다.

02 · 반응형

# 접두사를 붙이면 화면 크기별로 달라집니다

```
<div class="col-12 col-md-6 col-lg-4">카드</div>

col-12      기본(좁은 화면)  → 한 줄에 1개
col-md-6    768px 이상       → 한 줄에 2개
col-lg-4    992px 이상       → 한 줄에 3개

/* DAY 06의 미디어 쿼리를 클래스 이름으로 적는 셈입니다 */
```

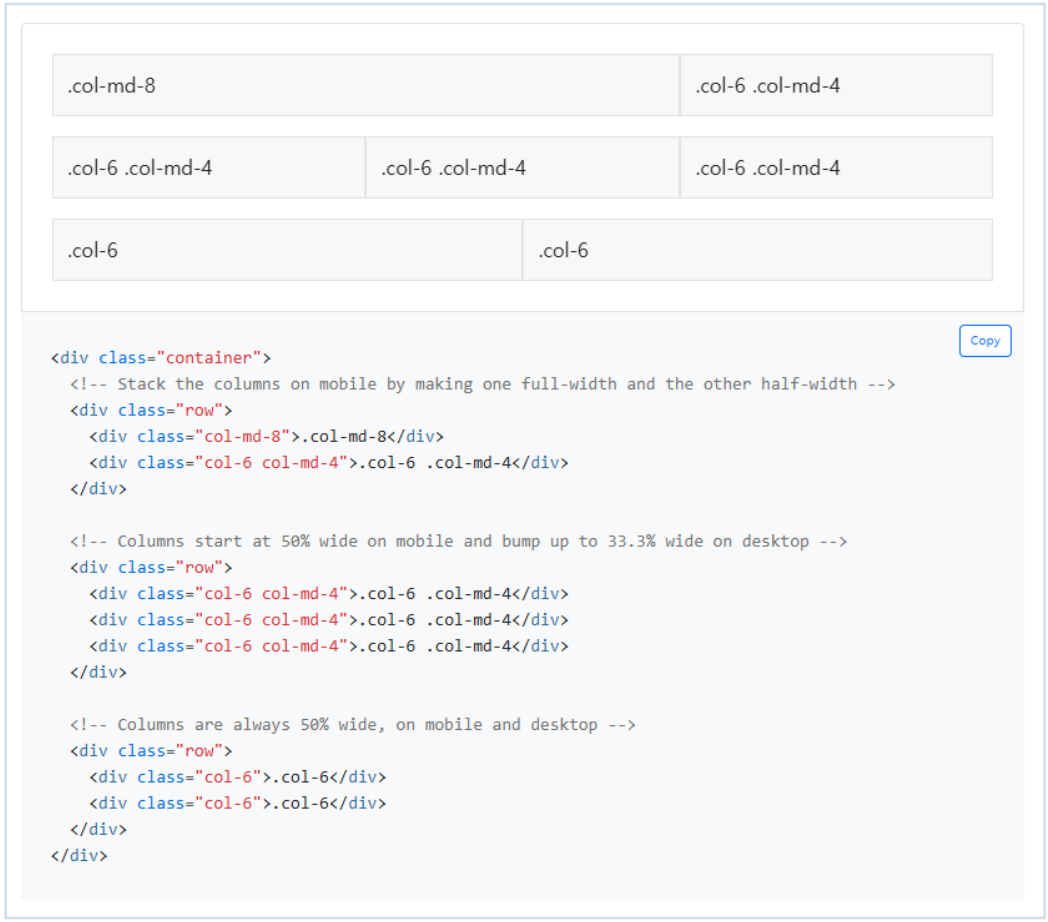
### 모바일 퍼스트입니다

접두사 없는 값이 기본이고
커질 때 덮어씁니다.
DAY 06에서 배운 방식과
똑같습니다.

### 미디어 쿼리를 직접 쓰지 않아도 반응형이 됩니다

DAY 06에서 손으로 만든 1열·2열·3열이 클래스 세 개로 끝납니다. 3열이 되는 폭은 1024px이 아니라 992px입니다.

# 접두사를 섞으면 화면마다 칸이 달라집니다




### 한 요소에 여러 개

col-6 col-md-4 처럼 붙이면
좁은 화면에서는 2열,
768px 이상에서는 3열이 됩니다.

### 미디어 쿼리를 안 씁니다

DAY 06에서 직접 쓰던 @media를
클래스 이름이 대신합니다.

02 · 나누는 방법

# 칸을 나누는 방법은 네 가지입니다

1. 균등 분할 — 숫자를 안 적으면 알아서 나눕니다
   `<div class="col">` `<div class="col">` `<div class="col">`

2. 한 칸만 지정 — 나머지는 남은 자리를 나눕니다
   `<div class="col">` `<div class="col-6">` `<div class="col">`

3. 내용 폭만큼 — 글자 길이에 맞춥니다
   `<div class="col-auto">`

4. 줄마다 몇 개인지 — 자식에 `col`을 안 써도 됩니다
   `<div class="row row-cols-3">`

### 숫자는 생략 가능

col 만 써도 균등하게 나뉩니다. 개수가 바뀌어도 알아서 맞춥니다.

**오늘 실습에서는 col-12 col-md-6 col-lg-4 형태를 씁니다**

02 · 분기점

# 접두사와 화면 폭

| 접두사 | 적용 폭 | 대략 어떤 기기 |
|---|---|---|
| 없음 | 0px 이상 | 휴대폰 세로 — 기본값 |
| sm | 576px 이상 | 휴대폰 가로 |
| md | 768px 이상 | 태블릿 세로 |
| lg | 992px 이상 | 태블릿 가로, 작은 노트북 |
| xl | 1200px 이상 | 데스크톱 |
| xxl | 1400px 이상 | 큰 화면 |

**실무에서는 md와 lg 두 개만 써도 대부분 해결됩니다**

# 칸 사이 간격과 줄 안 정렬

```
<div class="row g-3">           칸 사이 간격
<div class="row gy-4">          세로 간격만

<div class="row justify-content-center">    가로 가운데
<div class="row align-items-center">        세로 가운데

/* DAY 05의 Flexbox 속성 이름이 그대로 들어 있습니다 */
```

### g는 gutter

숫자는 0부터 5까지입니다.
g-3이면 1rem 정도입니다.

### 이름이 익숙한 이유

row는 안쪽이 Flexbox입니다.
DAY 05에서 배운 속성 이름을
클래스로 만든 것입니다.

02 · 중첩

# 칸 안에 다시 12칸을 만들 수 있습니다

```
<div class="row">
  <div class="col-8">          바깥의 8칸
    <div class="row">          그 안에서 다시 12칸
      <div class="col-6">왼쪽</div>
      <div class="col-6">오른쪽</div>
    </div>
  </div>
  <div class="col-4">사이드바</div>
</div>
```

### 안쪽도 12칸입니다

바깥이 8칸이어도
그 안은 다시 12칸으로
나뉩니다.

**col 안에 바로 col을 넣지 않습니다 — 사이에 row가 있어야 합니다**

실습 01

# Grid로 카드 세 개 배치하기

Day08/index.html을 만들고 Bootstrap을 연결한 뒤 Grid로 카드를 배치합니다. 화면을 줄여가며 1열·2열·3열로 바뀌는지 확인합니다.

```
<div class="container">
  <div class="row g-3">
    <div class="col-12 col-md-6 col-lg-4">
      <div class="card p-3">프로젝트 A</div>
    </div>
    <!-- 같은 칸을 두 개 더 -->
  </div>
</div>
```

### 확인할 것

- 좁으면 세로로 쌓인다
- 768px에서 2열이 된다
- 992px에서 3열이 된다

### 먼저 col-12 만 넣고

접두사를 하나씩 더해가며 확인합니다.

### 막히면

col이 row 안에 있는지 봅니다.

# 03

# 컴포넌트

Button · Card · Navbar · Form · Modal · 실습 02

# 컴포넌트를 쓰는 이유

01

## 빨라집니다

버튼 하나에 색·여백·모서리를
정하던 시간이 사라집니다.
클래스 이름만 적습니다.

02

## 일관됩니다

화면마다 다른 버튼이
생기지 않습니다.
전체가 한 벌처럼 보입니다.

03

## 동작이 검증돼 있습니다

키보드 이동과 포커스 처리가
이미 만들어져 있습니다.
다만 aria- 속성은
직접 붙여야 합니다.

### 다시 쓸 수 있게 미리 만들어 둔 UI 조각입니다

동작은 들어 있지만 aria-label 같은 속성은 공식 문서 예제를 그대로 따라 붙여야 합니다.

# 버튼은 두 클래스로 만듭니다

```
<button class="btn btn-primary">기본</button>
<button class="btn btn-outline-primary">테두리만</button>
<button class="btn btn-secondary btn-sm">작게</button>
<a href="#" class="btn btn-primary">링크도 버튼처럼</a>
<button class="btn btn-primary" disabled>누를 수 없음</button>

/* btn      = 버튼 모양                          */
/* btn-***  = 색  (primary·secondary·success  */
/*            danger·warning·info·light·dark) */
```

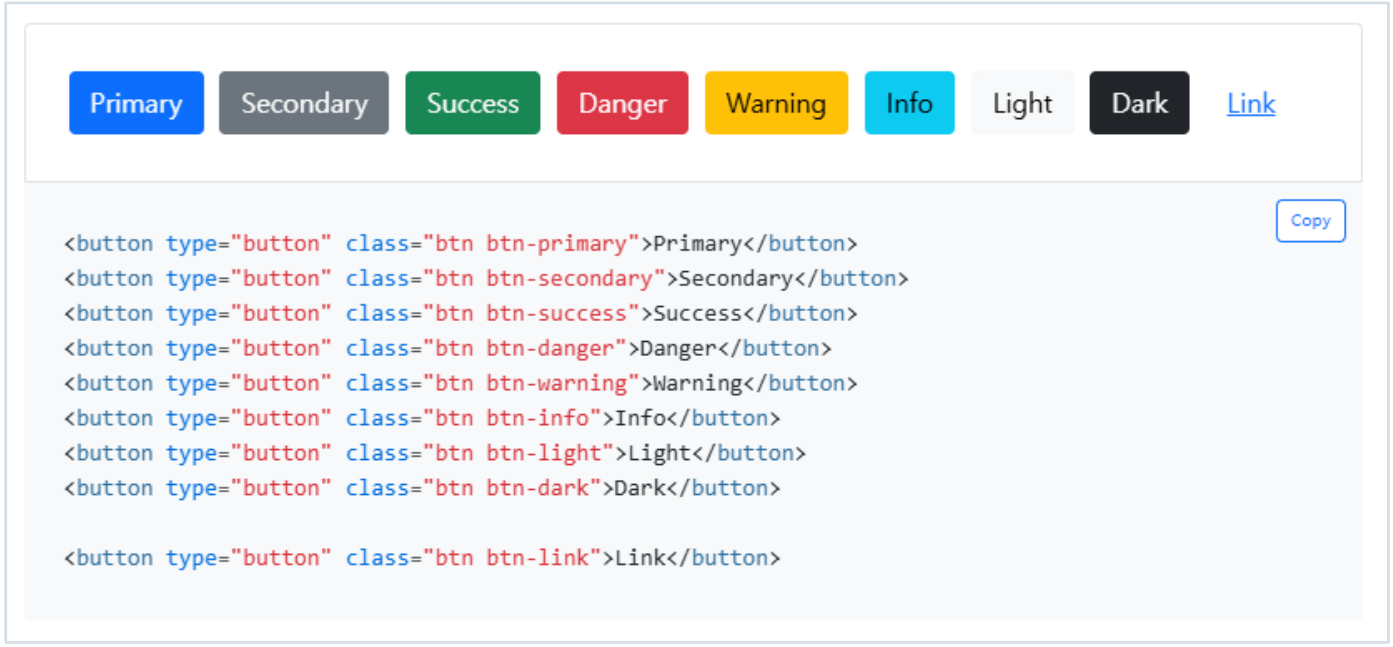
### 색 이름은 의미입니다

primary는 주 동작,
danger는 삭제처럼
되돌릴 수 없는
동작에 씁니다.

### 이 여덟 가지 색 이름은 다른 컴포넌트에서도 그대로 쓰입니다

alert-danger, text-primary, bg-light 처럼 이름 뒤에 붙습니다.

# 색 이름은 버튼에서 이렇게 보입니다




btn- 뒤에 색 이름을 붙입니다. 맨 오른쪽 btn-link는 링크처럼 보이는 버튼입니다.

**같은 색 이름이 배경색과 글자색에도 그대로 쓰입니다**

# Card는 안쪽 구조가 정해져 있습니다

```
<div class="card">
  <img src="me.jpg" class="card-img-top" alt="프로필">
  <div class="card-body">
    <h5 class="card-title">프로젝트 A</h5>
    <p class="card-text">간단한 설명이 들어갑니다.</p>
    <a href="#" class="btn btn-primary">보러 가기</a>
  </div>
</div>
```

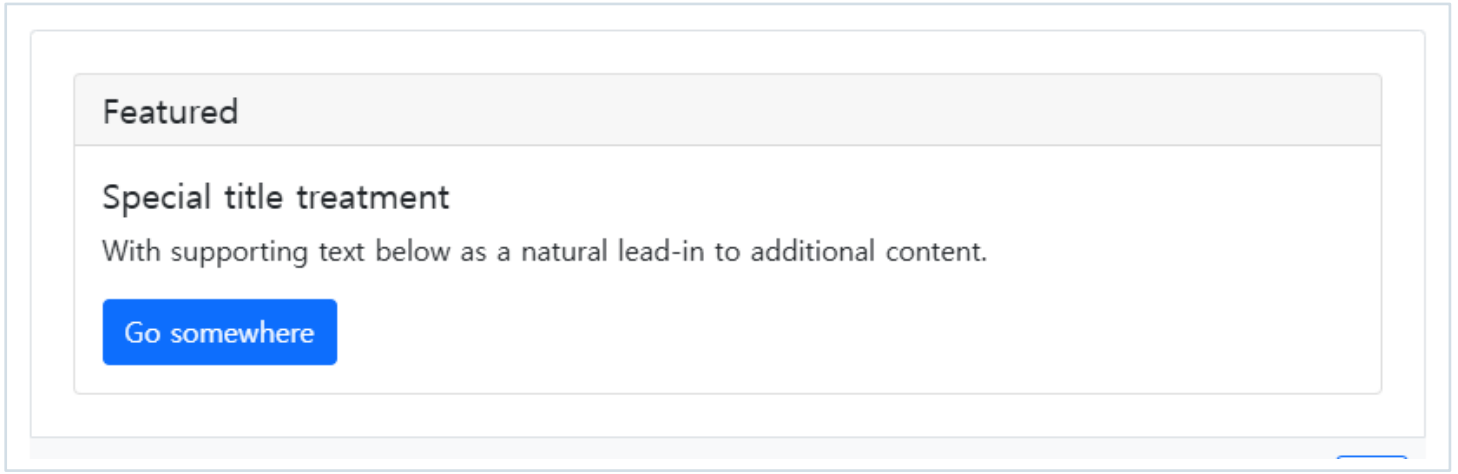
### card-body가 여백을

이것을 빼면 글자가
테두리에 붙습니다.

### Grid와 함께

col 안에 card를 넣으면
카드 목록이 됩니다.
실습 01이 그 구조입니다.

# Card는 이렇게 그려집니다




### 구조가 정해져 있습니다

card 안에 card-body,
그 안에 card-title과
card-text를 넣습니다.

### 높이 맞추기

한 줄에 놓인 카드의 높이는
h-100으로 맞춥니다.
DAY 09 미니 프로젝트에서
다시 씁니다.

# Navbar는 좁은 화면에서 접힙니다

```
<nav class="navbar navbar-expand-lg bg-light">
  <div class="container">
    <a class="navbar-brand" href="#">김코딩</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
            data-bs-target="#menu" aria-controls="menu"
            aria-expanded="false" aria-label="메뉴 열기">
      <span class="navbar-toggler-icon"></span></button>
    <div class="collapse navbar-collapse" id="menu">
      <ul class="navbar-nav">…</ul>
    </div>
  </div>
</nav>
```

### expand-lg

992px 이상에서 펼쳐지고
그 아래에서는 햄버거
버튼으로 접힙니다.

### 접히고 펼쳐지는 동작에는 JS가 필요합니다

body 끝의 bootstrap.bundle.min.js 가 없으면 버튼을 눌러도 아무 일이 없습니다.

# Form은 DAY 02의 구조에 클래스만 붙입니다

```
<div class="mb-3">
  <label for="email" class="form-label">이메일</label>
  <input type="email" class="form-control"
         id="email" name="email" required>
</div>

<button type="submit" class="btn btn-primary">보내기</button>

/* label의 for와 input의 id — DAY 03 접근성 그대로입니다 */
```

### 바뀌는 것은 모양뿐

type, name, required는
DAY 02에서 배운 그대로입니다

.

class만 더해집니다.

### Bootstrap을 써도 접근성 규칙은 그대로 지켜야 합니다

form-label 클래스가 label 태그를 대신해 주지는 않습니다.

03 · 검증 표시

# 입력이 잘못됐을 때 보여주는 방법

```
<input class="form-control is-invalid" id="email">
<div class="invalid-feedback">이메일 형식이 아닙니다</div>

<input class="form-control is-valid">
<div class="valid-feedback">확인했습니다</div>

/* DAY 02의 required·pattern 이 막아주는 것과 별개로 */
/* 무엇이 잘못됐는지 글로 보여주는 부분입니다 */
```

### 색만으로 알리지 않습니다

빨간 테두리만 있으면
색을 구분하기 어려운
사용자가 알 수 없습니다.
글을 함께 둡니다.

### 오류 메시지는 무엇이 잘못됐고 어떻게 고치는지 적습니다

# Modal은 버튼과 상자를 짝지어 씁니다

```
<button class="btn btn-primary" data-bs-toggle="modal"
        data-bs-target="#hello">열기</button>

<div class="modal fade" id="hello" tabindex="-1">
  <div class="modal-dialog"><div class="modal-content">
    <div class="modal-header">…</div>
    <div class="modal-body">내용</div>
  </div></div>
</div>
```

### id로 연결합니다

버튼의 data-bs-target과
상자의 id가 같아야
열립니다.

**안 열리면 JS가 연결됐는지, id가 같은지 두 가지를 봅니다**

# 짧게 쓰이는 컴포넌트들

```
<div class="alert alert-warning">저장되지 않았습니다</div>
<span class="badge bg-primary">NEW</span>

<ul class="list-group">
  <li class="list-group-item">항목 하나</li>
</ul>

<div class="spinner-border"></div>   불러오는 중 표시

<div class="carousel slide">…</div>  넘겨 보는 사진첩
```

### 색 이름이 또 나옵니다

alert-warning, bg-primary 처럼 버튼에서 본 여덟 가지 이름 그대로입니다.

### 이름 규칙을 알면 처음 보는 컴포넌트도 읽을 수 있습니다

컴포넌트 이름 + 하이픈 + 색 또는 크기. 대부분 이 형태입니다.

03 · 정리

# 오늘 쓴 컴포넌트 한눈에

| 컴포넌트 | 핵심 클래스 | JS 필요 |
|---|---|---|
| Button | btn + btn-primary | 아니요 |
| Card | card + card-body + card-title | 아니요 |
| Navbar | navbar + navbar-expand-lg + collapse | 예 — 접힘 동작 |
| Form | form-label + form-control | 아니요 |
| Modal | modal + data-bs-toggle + data-bs-target | 예 — 열고 닫기 |
| Alert·Badge | alert-*, badge bg-* | 아니요 |

**JS가 필요한 것은 움직이는 컴포넌트뿐입니다**

# 외우지 않고 문서에서 가져다 씁니다

컴포넌트는 종류가 많습니다. 필요할 때 찾아 복사하고 고치는 것이 정상적인 사용법입니다.

### 이 순서로 씁니다

- 문서에서 원하는 모양을 찾는다
- 예제 코드를 복사한다
- 내 내용으로 글자를 바꾼다
- 필요 없는 부분을 지운다

### 지울 때 조심할 것

- data-bs-* 속성은 동작에 필요합니다
- id와 target은 짝입니다
- 감싸는 div를 지우면 모양이 깨집니다

**문서** Bootstrap 5.3 — 컴포넌트 문서 (Navbar·Card·Modal 전부 여기)

**예제** Bootstrap Examples — 완성된 화면을 가져와 고쳐 쓸 수 있습니다

실습 02

# 내 소개 페이지를 Bootstrap으로 만들기

1챕터에서 만든 페이지를 Bootstrap 컴포넌트로 다시 구성합니다.
직접 쓴 CSS는 색과 글꼴 정도만 남깁니다.

### 해야 할 것

- Navbar로 상단 메뉴 만들기
- container·row·col로 본문 나누기
- Card로 작업 목록 만들기
- Form으로 연락처 입력칸 만들기
- Modal 하나 넣어보기

### 확인할 것

- 좁은 화면에서 메뉴가 접힌다
- 카드가 1열·2열·3열로 바뀐다
- 모달이 열리고 닫힌다
- label과 input이 연결돼 있다
- style.css가 짧아졌다

**다 되면 커밋하고 push합니다 — GitHub 저장소 화면에서 Day08 폴더를 확인**

# 04

# Utility Class

간격 · 색 · 글자 · 정렬 · 언제 쓰나

04 · 간격

# 여백은 글자 하나와 숫자로 정합니다

```
m = margin      p = padding
t 위  b 아래  s 왼쪽  e 오른쪽  x 좌우  y 위아래
0 1 2 3 4 5     0 · 0.25 · 0.5 · 1 · 1.5 · 3 rem

<div class="mt-3">     위쪽 여백 1rem
<div class="px-4 py-2">좌우 1.5rem, 위아래 0.5rem
<div class="mb-0">     아래 여백 없음
<div class="mx-auto">  좌우 auto — 가운데 정렬
```

### s와 e인 이유

left·right 대신
start·end를 씁니다.
왼쪽에서 오른쪽으로 읽는
언어에서는 왼쪽과
오른쪽이 됩니다.

### 규칙 하나로 수십 개의 클래스가 만들어집니다

mt-3, pb-5, mx-auto — 외우는 것이 아니라 조합하는 것입니다.

# 자주 쓰는 나머지 유틸리티

```
text-primary  text-body-secondary  text-danger  글자색
bg-light  bg-dark  bg-white                    배경색

text-center  text-end                          정렬
fw-bold  fs-4  lh-base                         굵기·크기·줄간격

rounded  rounded-circle  shadow-sm  border     모양

<p class="text-body-secondary text-center fs-5">설명 문구</p>
```

### 색 이름이 같습니다

btn-primary의 primary와 text-primary의 primary는 같은 색입니다.

### 전부 외우지 말고 필요할 때 문서에서 찾습니다

# 화면 크기별로 보이고 숨깁니다

```
<div class="d-none d-md-block">넓은 화면에서만 보임</div>
<div class="d-block d-md-none">좁은 화면에서만 보임</div>

<div class="d-flex justify-content-between align-items-center">
  Flexbox도 클래스로 쓸 수 있습니다
</div>

/* DAY 05의 display와 DAY 06의 미디어 쿼리가 합쳐진 형태입니다 */
```

### 간격에도 붙습니다

mt-md-5 처럼
접두사를 넣습니다.

### 지금까지 배운 CSS 개념이 이름만 바뀌어 들어 있습니다

새로운 개념을 배우는 것이 아니라 이미 아는 것을 짧게 적는 방법을 배우는 중입니다.

# 아이콘도 클래스로 넣습니다

```
<!-- head 에 한 줄 더 -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3
      /font/bootstrap-icons.css" rel="stylesheet">

<i class="bi bi-github"></i>
<i class="bi bi-envelope"></i>

/* 글자처럼 다뤄지므로 색과 크기를 CSS로 정합니다 */
```

### 이름은 문서에서

아이콘 검색 페이지에서
이름을 복사해 붙입니다.

### 의미가 있는 아이콘에는 대체 텍스트를 함께 둡니다

아이콘만 있는 링크는 화면낭독기가 읽을 것이 없습니다. DAY 03에서 배운 그대로입니다.

04 · 판단

# 언제 유틸리티를 쓰고 언제 CSS를 쓸까

유틸리티가 나은 경우

## 한두 군데의 조정

이 카드만 여백을 조금 더,
이 문단만 가운데 정렬.
CSS 파일을 열지 않아도 됩니다.

CSS가 나은 경우

## 반복되는 규칙

모든 카드에 같은 그림자,
사이트 전체의 글꼴과 색.
한 곳만 고치면 전부 바뀝니다.

**클래스가 열 개씩 붙기 시작하면 CSS로 옮길 때입니다**

HTML이 읽기 어려워지는 순간이 기준입니다.

04 · 정리

# 오늘 쓴 클래스는 여기서 다시 찾습니다

문서를 즐겨찾기에 넣어두고 필요할 때 검색해서 쓰는 것이 정상적인 사용법입니다.

**문서** Grid System — 12칸과 반응형 접두사

**문서** Spacing — 간격 유틸리티

**문서** Breakpoints — 분기점 정리

**문서** Flex — 정렬 유틸리티

### 검색할 때는 반드시 버전을 함께 씁니다

「bootstrap 5 navbar」처럼 5를 붙이지 않으면 3·4버전 답이 섞여 나옵니다.
클래스 이름이 달라 그대로 붙이면 동작하지 않습니다.

점검

# Bootstrap에서 자주 나오는 세 가지

!

### 스타일이 하나도 안 먹어요

CDN 링크가 head에
없거나 주소에 오타가
있는 경우입니다.

!

### 모달·메뉴가 안 움직여요

body 끝의 JS 파일이
빠진 경우입니다.
bundle 쪽을 씁니다.

!

### 검색한 코드가 안 돼요

3·4버전 코드입니다.
클래스 이름이 다릅니다.
5버전 문서를 봅니다.

**Bootstrap은 CSS를 대신 써 주는 것이지 CSS를 몰라도 되게 해주지 않습니다**

1챕터에서 직접 써봤기 때문에 클래스가 무엇을 하는지 읽을 수 있습니다.

WRAP UP

# 오늘 정리

## 개념

- Bootstrap은 미리 만들어 둔 CSS 묶음이다
- CDN 링크는 head, 스크립트는 body 끝에 넣는다
- 내 CSS는 Bootstrap 링크보다 뒤에 둔다
- Grid는 12칸이고 접두사로 반응형이 된다
- 컴포넌트는 문서에서 복사해 고쳐 쓴다
- 유틸리티는 한 가지만 바꾸는 작은 클래스다

## 직접 쓸 수 있어야 하는 것

- CDN으로 Bootstrap 연결하기
- container·row·col로 배치하기
- col-12 col-md-6 col-lg-4 쓰기
- Navbar·Card·Form·Modal 가져다 쓰기
- 간격 유틸리티로 여백 조정하기
- d-none d-md-block으로 감추기

NEXT

## DAY 09

# 클론코딩 미니 프로젝트

1챕터의 HTML·CSS와 오늘의 Bootstrap을 모두 써서
자기소개 페이지를 완성하고 발표합니다.

수고하셨습니다.